

AK VisionFlow — Phase 0.1

Existing System Audit & Feature Mapping — Detailed Developer Specification

Master Phase: Phase 0 — Universal Architecture & Refactoring

Sub-Phase: 0.1

Purpose: Establish an evidence-based baseline of the existing product before universalization.

Important: Phase 0.1 is an audit and mapping phase. Do not perform a major rewrite merely because the product is becoming universal. First inspect, document, classify and identify the safest refactoring path.

1. Objective

Create a complete inventory of the current AK VisionFlow web, backend, database and Owner Android capabilities.

Map each capability to its UI, API/service, database entities, permissions, tenant/branch behavior and tests.

Classify each capability as Universal Core, Universal Module, Industry Module, Configuration, Platform/Infrastructure, Mobile or Integration.

Identify hard-coded Optical/Medical assumptions and the changes required to support multiple industries.

Produce the factual baseline required by Phase 0.2–0.5.

2. Product Direction

AK VisionFlow is being repositioned as a multi-tenant, modular Business Operating System for SMBs.

Optical, Eye Clinic and Medical Store functionality must be preserved as specialized industry packs, not treated as the permanent product boundary.

New businesses should eventually select an industry during onboarding and receive relevant modules, workflows, fields and dashboards automatically.

Businesses must later be able to enable additional modules/industry capabilities without creating a separate application.

3. Mandatory Evidence to Inspect

Repository/source: backend, frontend, Android, migrations, tests, configuration and existing documentation.

Database: tables, relations, indexes, constraints, enums, tenant/branch fields, migrations and seed/test data. No unauthorized production writes.

Backend/API: all routes, controllers/services, validation, authentication, authorization, business rules, jobs and integrations.

Web UI: navigation, screens, forms, dashboards, reports, filters, role visibility and customer-facing documents.

Android: authentication, Home, Analytics, Alerts/Push, AI Advisor, Profile, Logout and API dependencies.

Tests: map every suite to the business capabilities it actually verifies.

4. Existing Capability Baseline to Verify

The existing audit explicitly reports backend coverage for products, customers, suppliers, categories, branches, sales including reversal, purchases/receiving, payments, optical orders, stock/inventory, expenses and the existing AI Recommendation Center.

The Owner Android audit reports local verification of authentication/security, dashboard/analytics, alerts/push and AI Advisor, with 332/332 backend tests passed and Android build/unit tests/lint successful.

Tenant isolation, RBAC and cross-tenant testing were reported as verified. Existing web-system regression tests passed with zero regressions.

Deployment status in the September 16, 2026 audit was incomplete: code had not been pushed, a mobile production migration was pending, production API/push configuration was pending, and manual device testing was not verified. Recheck these items rather than assuming they remain current.

5. Required Feature Inventory

Create one master row per feature. Required columns: ID; Domain; Feature; Current Status; UI Location; API/Service; DB Entity; Tenant Scoped?; Branch Scoped?; Roles; Tests; Classification; Reuse Decision; Gap; Risk; Dependency; Recommended Phase; Evidence/Code Location.

Status values: Existing/Complete; Existing/Partial; Existing/Legacy; Existing/Needs Refactor; Missing; Unknown—Requires Inspection.

Every finding must include evidence: route, table/migration, test name, component/file, screen or other verifiable location.

6. Domains That Must Be Mapped

Identity & tenancy: company, tenant, branch, warehouse, users, roles, permissions and authentication.

Master data: products, services, categories, brands, units, customers, suppliers, prices and taxes.

Commerce: quotations, sales orders, invoices, POS, payments, receipts, returns, refunds and credit/debit notes.

Procurement: suppliers, purchase orders, receiving, purchase invoices, supplier payments and returns.

Inventory: stock, movements, transfers, adjustments, valuation, low-stock, batch/serial/expiry where applicable.

Finance: existing financial summaries and transaction links; document exactly what accounting exists today before Phase 2.

Operations: expenses, staff, appointments, projects, tasks, documents, helpdesk and other current capabilities.

Optical/Healthcare: prescriptions, frames/lenses, optical orders, fitting/lab, delivery, patient/clinic workflows.

AI and notifications: Recommendation Center, Owner AI Advisor, push, email, SMS, WhatsApp and integrations.

Reporting and mobile: current KPIs, dashboards, filters, drill-downs and Owner Android dependencies.

7. Classification Rules

Universal Core: concepts broadly needed by businesses—customers, suppliers, products/services, sales, purchases, payments, expenses, branches, users, stock, invoices and core reporting.

Universal Module: broadly useful but optional—CRM, POS, HR, payroll, projects, documents, helpdesk, appointments, subscriptions.

Industry Module: strongly sector-specific workflows—eye prescription, optical fitting/lab, medicine batch/expiry, restaurant kitchen/table workflows.

Configuration: custom fields, categories, statuses, workflows, approval rules, dashboard widgets and document templates.

Platform/Mobile/Integration: infrastructure, mobile capabilities and third-party connections.

When uncertain, mark the finding as uncertain and record the evidence required. Do not force a classification.

8. Database Audit

For every major entity record purpose, primary key, tenant key, branch key, relations, status, audit fields, unique constraints, indexes, deletion behavior and migration history.

Identify hard-coded Optical/Medical assumptions in columns, enums, mandatory fields, UI labels and business rules.

Identify duplicate concepts and candidates for a universal entity model.

Record tables as: reusable as-is; reusable with extension; industry-specific; legacy; unclear.

No destructive schema change is part of Phase 0.1.

9. API & Business Logic Audit

Create an endpoint inventory containing method, route, authentication, role, tenant scope, branch scope, validation, response, service/database dependencies, read/mutation behavior and tests.

Identify duplicated business logic between web and mobile. Business rules should remain consistently enforced by backend services.

Identify APIs that assume a single industry or fixed Optical workflow.

Record security issues such as missing object-level authorization or branch checks for remediation/escalation.

10. UI/UX Audit

Inventory every screen and navigation item. Mark universal, module-specific, industry-specific and role-specific items.

Map dashboard widgets, KPI calculations, filters and drill-downs.

Find hard-coded company names, logos, colors, Optical labels and clinic-specific assumptions.

Record all current branding locations to support the future Tenant Branding Engine.

Do not redesign the UI in this phase; create the evidence-backed refactoring backlog.

11. Owner Android Audit

Map every Owner App screen to backend routes/services and identify which capabilities are universal Owner Command Center functions.

Preserve the verified security baseline: owner-only mobile access, token isolation, secure token storage, production HTTPS and tenant isolation.

Track the audit limitation that physical/on-device testing remained unverified because the emulator could not start with hardware virtualization disabled.

12. Test Coverage Audit

Create a feature-to-test matrix separating unit, integration/API, security, UI and manual testing.

Identify gaps in cross-tenant and cross-branch access, role escalation, stock integrity, transaction reversal, payment linkage, concurrency and migration safety.

Do not remove or weaken existing tests during universalization.

13. Odoo / QuickBooks Gap Mapping

Compare capabilities rather than implementation details. The purpose is to identify business-critical gaps for the intended alternative positioning.

Odoo-oriented areas: accounting, sales, purchase, inventory, POS, CRM, expenses, HR, projects, documents, helpdesk, appointments, subscriptions and future eCommerce/manufacturing.

QuickBooks-oriented areas: Chart of Accounts, double-entry accounting, A/R, A/P, invoicing, expenses, banking, reconciliation, taxes, financial reports, multi-currency and statements.

Mark each capability Existing / Partial / Missing / Not Yet Planned / Industry-Specific. Do not claim parity without implementation and testing.

14. Required Deliverables

1. Current Feature Inventory.
2. Feature Classification Matrix.
3. Database Entity Map and hard-coded industry dependency list.
4. API/Business Logic Map.
5. Web UI/Navigation Map.
6. Owner Android dependency map.
7. Test Coverage Matrix and gaps.
8. Odoo/QuickBooks capability gap matrix.
9. Refactoring backlog with priority, risk and dependency.

10. Phase 0.2 input package.

15. Risk Levels

Critical: security, tenant isolation, data integrity, accounting integrity or destructive migration risk.

High: architecture constraints likely to cause major rework, duplicated business logic or hard-coded single-industry assumptions.

Medium: incomplete feature, reporting or automation issue that does not block architecture.

Low: cosmetic or minor usability issue.

16. Developer Rules

Do not begin a major refactor before completing the evidence-based inventory.

Do not remove working Optical/Medical functionality.

Do not make production database writes without explicit authorization.

Do not invent undocumented features; use Unknown—Requires Inspection where necessary.

Do not treat UI presence as proof of backend correctness.

Do not treat passing tests as proof of complete business coverage.

Tenant isolation, branch isolation and RBAC are non-negotiable.

17. Definition of Done

Every major existing capability has a traceable UI → API/service → database → test map.

Every capability has an evidence-backed classification.

All hard-coded Optical/Medical assumptions are documented.

All major Odoo/QuickBooks-alternative gaps are documented.

Security, tenancy, branch and data-integrity risks are prioritized.

Owner Android dependencies and constraints are mapped.

A dependency-aware refactoring backlog is ready for Phase 0.2 onward.

Client/product-owner approval is obtained before architecture implementation.

18. Phase 0.1 Output

At completion, management must be able to answer: What do we have today? What is reusable? What is Optical/Medical-specific? What is missing for a universal ERP/accounting platform? What must be refactored? What must remain unchanged? What should be built next?

The output is not new functionality. It is a reliable technical/product baseline that prevents unnecessary rewrites and makes later phases measurable.

19. Sample Feature Mapping Format

Feature	Evidence	Classification	Future Treatment
Products	DB + API + UI + tests	Universal Core	Reusable product/service engine
Customers	DB + routes + screens	Universal Core	Universal customer model
Optical Orders	Routes + tables + screens	Industry Module	Optical Industry Pack
Inventory	Stock routes + movement data + tests	Universal Core/Module	Universal inventory engine
AI Recommendation Center	Routes/services/tests	AI Layer	Generalize across modules

Feature	Evidence	Classification	Future Treatment
Owner Analytics	Android screens + mobile APIs	Mobile Platform	Universal Owner Command Center

The sample classifications are starting points only. The developer must confirm them against the actual repository/database.

20. Approval Checklist

- Source repository, backend, frontend and Android inspected.
- Database schema and migrations inspected.
- APIs and services mapped.
- UI/navigation mapped.
- Tests mapped to capabilities.
- Tenant/branch/RBAC behavior documented.
- Optical/Medical hard-coding identified.
- Odoo/QuickBooks gaps documented.
- Refactoring risks/dependencies documented.
- Phase 0.2 inputs prepared.
- Client/product-owner approval obtained.